Configuring Default Routes

Use `route(8)` to configure the default route for each protocol.

```
# route add default 192.0.2.1
add net default: gateway 192.0.2.1
```

An IPv6 default route is almost identical, but you must add the `-inet6` modifier.

```
# route add -inet6 default 2001:db8:0:12::1
add net default: gateway 2001:db8:0:12::1
```

Once you add IP addresses and default routes to your host, you should be able to reach the rest of your network and the Internet. Now let's see how to make those changes across reboots.

Using Dynamic Configuration

To have OpenBSD get an IPv4 address from a DHCP server, run `dhclient(8)` and give it the name of the interface you want to configure.

```
# dhclient fxp0
```

`dhclient` gets an IP address, overwrites */etc/resolv.conf*, and configures the default route.

For IPv6, run `rtsol(8)` instead.

```
# rtsol fxp0
```

Remember that IPv6 autoconfiguration will not configure your resolver. You'll need to piggyback off your IPv4 DNS servers or manually configure */etc/resolv.conf*.

Configuring the Network at Boot

While `ifconfig(8)` is fine for changes on the fly, your system should configure its interfaces correctly at boot, including any aliases on the interface, any routes added when the interface comes up, and so on.

Each interface has a configuration file, */etc/hostname.interfacename*, generically called *hostname.if*. The `fxp0` interface on my desktop uses a configuration file */etc/hostname.fxp0*, my wireless interface `wpi0` uses */etc/hostname.wpi0*, and so on. At boot, OpenBSD's */etc/netstart* script reads all of the *hostname.if* files and, if it finds a matching physical interface or can create a matching logical interface, it configures the interface accordingly.

To configure an interface's IPv4 address, enter a line in *hostname.if* in this format:

```
inet ipaddress netmask broadcastaddress ifconfig-options
```

The broadcast address and options are optional. To use options but not specify a broadcast address, use `NONE` for the broadcast address. You can also use a slash for the netmask instead of the decimal equivalent.

Similarly, add an IPv6 address with the following:

```
inet6 ipv6address/prefix ifconfig-options
```

To give `fxp0` the IPv4 address of 192.0.2.226 255.255.255.240 and the IPv6 address of 2001:db8:0:12::2/64 at boot, use the following in */etc/hostname.fxp0*:

```
inet 192.0.2.226 255.255.255.240 NONE description 'top card'
inet6 2001:db8:0:12::2/64
```

Here, I also define an interface description that will show up in `ifconfig` output.

To create an IP address alias at boot, use the `alias` keyword in *hostname.if*.

```
inet alias 192.0.2.230/32
inet6 alias 2001:db8:0:12::3/64
```

To run a command when the interface comes up, put an exclamation point in front of the command. Any commands run must be available on the root partition (for example, in */bin* or */sbin*). This feature is most commonly used for routing, but you could use other commands as well.

```
!route add 192.0.2.128/25 192.0.2.2
```

To configure an interface dynamically, via DHCP (IPv4) or `rtsol` (IPv6), put the string `dhcp` or `rtsol` on a line by itself.

```
dhcp
rtsol
```

Anything that's not formatted as shown here is passed unedited to `ifconfig(8)`. For example, to run a specific `ifconfig` command, put the arguments on their own line in *hostname.if*.

```
description 'lower card'
```

If you simply want to activate a card, but not configure it, use the word `up` on a line by itself to activate the interface.

```
up
```

And remember, you can test *hostname.if* changes with */etc/netstart*, specifying an interface name if appropriate, like so:

```
# /bin/sh /etc/netstart fxp0
```

Not including the interface name reconfigures all interfaces on the system.

Trunking

Servers can have redundant hard drives, power supplies, and so on. OpenBSD supports redundant network connections by combining multiple Ethernet links into a single virtual link, or *trunk*. You might also know of this as *link aggregation*, *network adapter teaming*, or *bonding*.

NOTE *Cisco people know of* trunks *as Ethernet links that support multiple concurrent VLANs. Most vendors, including OpenBSD, don't use the word* trunk *in that way. OpenBSD supports sending multiple VLANs over a single link outside the* `trunk(4)` *functionality.*

Link Aggregation Protocols

To use multiple physical links as a single large link, you need a way to distribute traffic between the links. OpenBSD supports five different ways to distribute frames between trunk members, though not all will work in all environments. For a complete list see `trunk(4)`, but the protocols I recommend for real-world use are Link Aggregation Control Protocol (LACP), roundrobin, and failover. LACP is the industry standard for link aggregation. The physical interfaces are bonded into a single virtual interface with roughly the same bandwidth as the sum of the individual interfaces. LACP is very fault-tolerant, and just about every high-end managed switch should support it. If your switch supports LACP, use it, but you must configure LACP on the switch ports before this kind of trunk will pass traffic.

In the roundrobin method, OpenBSD sends frames across the trunk's active connections using a roundrobin scheduler. The trunk accepts incoming packets on any port, and a roundrobin scheduler rotates between the trunk connections, with error and edge handling added on top. Roundrobin trunks don't need any special switch configuration; they just need two ports in the same VLAN.

In the case of failover, OpenBSD sends and receives all traffic over the first port in the trunk, and if that port fails, it switches to another active port. The failover method doesn't give you any additional bandwidth, but requires absolutely no support from the switch, and it even works on old-fashioned hubs.

Trunk Configuration

As an example, let's configure ports `em0` and `em1` into failover trunk `trunk0`. The underlying ports have never been configured before, so begin by activating these interfaces without any configuration.

```
# ifconfig em0 up
# ifconfig em1 up
```

Now create the failover trunk with `ifconfig(8)` and add these ports to it to make the `trunk0` interface usable.

```
# ifconfig trunk0 trunkproto failover
# ifconfig trunk0 trunkport em0
# ifconfig trunk0 trunkport em1
```

You could do this all in one long `ifconfig` command, but I find simpler, shorter commands easier to understand when learning.

Assign the interface an IP address just as you would a physical interface, and add a default gateway to your system.

```
# ifconfig trunk0 192.0.2.8 netmask 255.255.255.0
# route add default 192.0.2.1
```

You should now have a failover trunk attached to your local network. To configure another trunk protocol, just specify the desired trunk protocol when you create the trunk. You'll find a complete list of trunk protocols in `trunk(4)`.

Trunks at Boot

Configure your trunk in */etc/hostname.if*. For example, suppose you need to edit *hostname.em0*, *hostname.em1*, and *hostname.trunk0*. Both of the *em* files contain only a single word:

```
up
```

This activates the interfaces, but does no configuration.

hostname.trunk0 is more complicated.

```
trunkproto failover
trunkport em0
trunkport em1
192.0.2.8 netmask 255.255.255.0
```

You can put all of these entries in a single line, just as you can configure the trunk with a single `ifconfig` command, but again, I find multiple lines easier to read and understand.

Your trunk should now start at boot.

Note that trunks do not necessarily need to consist of interfaces that use the same type of physical medium. If you're feeling adventurous, you could try to replicate what some OpenBSD developers and users have been known to do: Trunk together a wired and a wireless network interface, and have all your connections survive (graceful failover, remember?) when you yank the plug out of your Ethernet port, or if you plug yourself back in and take your access point down for maintenance.

VLANs

VLANs are a way to get multiple Ethernet segments on a single piece of wire. You'll sometimes see this referred to as *802.1q*, *tagging*, or a combination of these terms.

In OpenBSD terms, one wire can carry multiple networks, and by configuring an additional interface, you can talk to those additional networks as if they had their own private wire. The wire can still carry only so much data, however, so all VLANs and the regular network (or *native VLAN*) that share the wire share the same pool of bandwidth.

VLAN frames that arrive at your network card are like regular Ethernet frames, with an additional header before the Ethernet frame that says "This is part of VLAN number such-and-such." Each VLAN is identified by a number. VLAN number 1 is usually the native VLAN—the VLAN that arrives without any tagging whatsoever. For convenience, I'll use the word "tagged" to describe how the VLAN is delivered to your host.

How would you use VLANs in OpenBSD? Perhaps you have a network divided into multiple Ethernet segments, such as outside the firewall, server area, and desktop clients. Or you might have one OpenBSD host that needs direct access to all of these segments. You could route all of these networks over a single physical wire. You might eventually hit bandwidth problems, but if you're pushing more than 1Gpbs through your server, you can afford a second network card.

Configuring Switches

You must configure your switch to send the VLANs to your OpenBSD box as 802.1q or tagged, depending on the switch's syntax. Cisco uses *802.1q*, HP's Procurve switches use *tagged*, and other vendors use whatever their prejudices dictate. There are dozens of different syntaxes to do this, so I won't give a specific example. If the switch can't send tagged VLANs to your server, you cannot use VLANs.

Configuring VLAN Devices

OpenBSD creates `vlan(4)` interfaces upon request. To create the device, you need to know which physical device you want to attach the VLAN to and the number of the VLAN you're expecting.

Create the `vlan` interface with `ifconfig`.

```
# ifconfig vlanX vlan vlan# vlandev interface
```

I number my `vlan` interfaces after the VLAN number they're used for. (You could create interface `vlan0` and attach it to VLAN 3, but that's too confusing for my feeble brain.) If you don't specify the VLAN number, OpenBSD assigns the VLAN number from the number on the interface.

For example, here I create interface `vlan3` and use it to access VLAN 3 over interface `fxp0`.

```
# ifconfig vlan3 vlandev fxp0
```

That's really all there is to it. Now you can use `ifconfig` to display your new interface:

```
$ ifconfig vlan3
vlan3: flags=48843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST,INET6_PRIVACY> mtu 1500
        lladdr 00:16:36:c0:58:a5
        priority: 0
        vlan: 3 parent interface: fxp0
        groups: vlan
        status: active
        inet6 fe80::216:36ff:fec0:58a5%vlan3 prefixlen 64 scopeid 0x7
```

This looks exactly like any physical interface, and from your point of view, it is. You can add IP addresses just as you would to any other interface, assign routes, and get on with your life.

Configuring VLANs at Boot

To configure a VLAN interface at boot time, create a *hostname.if* file for it. For example, here's the contents of a */etc/hostname.vlan3* that creates the `vlan3` interface demonstrated in the previous section, assigns it to VLAN 3, and configures it automatically for both IPv4 and IPv6:

```
vlandev fxp0
dhcp
rtsol
```

OpenBSD should find this file at boot and create the interface according to your commands.

IPv6 Over Tunnels

Let's say you've taken my badgering to heart and decided to experiment with IPv6, but your ISP doesn't offer IPv6. How can you play with IPv6 when all you get is an IPv4 feed?

Many companies offer a free IPv6 tunnel service, where they will route you through an IPv6 tunnel over IPv4. They will even give you an IPv6 /64 at no charge, so you can configure your home network for IPv6.

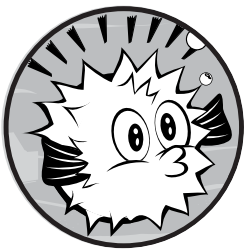
While I generally avoid recommending vendors in this book, I do recommend Hurricane Electric's IPv6 tunnel service at *http://www.tunnelbroker.net/*. Its web interface is intuitive, and it even provides configurations for OpenBSD clients.

You should now have some understanding of managing IPv4 and IPv6 on OpenBSD. While your brain is recovering from all this stuff, we'll turn to the topic of managing add-on software in OpenBSD.

13

SOFTWARE MANAGEMENT

Blowfish is solid,
but the third-party software?
Easy road to ruin.



Most people don't use an operating system; they use software, which runs atop an underlying operating system. No matter how robust an operating system is, it's useless without applications.

Many commercial operating systems include hundreds or thousands of small programs: games, desktop toys, and everything from fancy-looking clocks to disk scrubbers and web browsers. Most users never touch most of these programs, but the programs take up disk space (and possibly other resources) just the same. Every program drags along some amount of infrastructure, and all of this software can cause any number of problems.

Unlike many other operating systems, OpenBSD deliberately includes relatively little software in the default installation. You get exactly what you need to provide the infrastructure for software, and nothing more. While a traditional UNIX or Unix-like system includes compilers, games, and man pages, you don't even need to install these when installing OpenBSD! Even

if you install everything included in OpenBSD, it will have far less software than any commercial operating system. That's because almost everything is considered an add-on package.

The advantage to this sparseness is that you know exactly what's on the system, which simplifies debugging. A random shared library from a program you've never used won't break your programs. The downside is that you need to think a bit to decide exactly what you do want to include, and you'll need to install those programs. OpenBSD makes installing software as easy as possible through the ports and packages system, which is introduced in this chapter. But first, let's take a look at building software.

Making Software

Building software is complicated because source code must be very specifically processed to create a program that works—let alone a program that works well! The `make(1)` program makes building software easily reproducible, so that a program can be built exactly as the software author intends. `make` takes its instructions from a configuration file, or *makefile*, which tells `make` exactly how to build a program from source. You don't need to know the internals of a makefile, so we're not going to dissect one here.

A makefile includes one or more targets and a set of instructions to carry out. For example, typing `make install` tells `make` to check the makefile for a procedure called `install` and, if found, to execute it. A target's name usually relates to what `make` should be doing. The `make install` process, for example, usually installs the software built by previous steps. You'll find targets to install, configure, and uninstall most software, and `make` handles a huge variety of functions, some of which far outstrip the creators' original intentions.

Source Code and Software

Source code is the human-readable instructions for building the actual machine code that makes up a program. You've probably already been exposed to some form of source code; if not, go look at a few files under */usr/src* (assuming, of course, that you installed the source code as I recommended back in Chapter 3). While you don't have to read source code, you should be able to recognize it.

Once you have a program's source code, you build (or compile) the program on the type of system on which you want to run it. (Building software for a foreign platform via cross-compiling demands that you know much more about building software, and is not always possible.) If the software was written for an operating system sufficiently similar to what you're building it on, you'll create a working program. If the operating system differs too much, either the build fails or the resulting software won't run. Once you've built the software successfully on your system, you can copy the

resulting program (or *binary*) to other systems on the same platform, with the same operating system version and supporting software, and expect it to run.

Some programs are sufficiently well written that you can compile them on many different platforms. A few programs specifically include support for widely divergent platforms. For example, you can compile the Apache web server on Windows, NetWare, and Unix-like platforms by typing `make install`. This is quite uncommon, however, and takes considerable effort on the part of the software authors. It also makes the code more complex, and supporting all these operating systems means that it cannot easily support all the features that make those operating systems special. (But note that the ability to *build* software on a variety of platforms doesn't necessarily mean that it *runs* well on all of those platforms.)

Generally speaking, if you can build a program from source, it usually runs. A sufficiently experienced sysadmin can use the source code and error messages to learn why a program won't build, or why it builds but doesn't run. In many cases, the problem is simple and can be fixed with minimal education.[1] This is one reason why access to source code is important.

Back when every sysadmin was a programmer, debugging software builds absorbed major portions of the sysadmin's time. Every Unix-like platform was slightly (or wildly) different. To build programs, sysadmins needed to understand their platform, the software's original platform, and the differences between the two. The duplication of effort to build common programs was truly horrendous. Tools such as `autoconf` and `configure` were intended to help simplify this problem, but these programs just paper over the underlying problems. Building many software packages requires much more time running `configure` scripts than they need to actually compile.

The OpenBSD ports and packages system removes all this pain.

The Ports and Packages System

Ports are a mechanism for reproducibly and consistently building software on OpenBSD. *Packages* are precompiled ports for a specific OpenBSD version and platform. Packages install quickly and easily, and are recommended by the OpenBSD folks. Installing from a port takes more time and effort, but can be customized for your environment or server.

The basic idea behind the ports system is that if source code must be modified or tweaked to build or run on OpenBSD, the modification process should be automated. If you need other software to build this program from source code or run it, those dependencies should be automatically used. If you record exactly which files the software installs, you can easily uninstall it. And if you have all of these things, you can pick up the software and install it on any similar OpenBSD system.

1. In the IT industry, "minimal education" means a willingness to dig in and figure it out, plus a few years of college or professional experience; access to programming textbooks or other educational materials; or a whole lot of youth, stubbornness, and motivation.

Packages are the installable files produced by the ports system. You can install packages over the network, either from your own package repository or from an OpenBSD mirror site. But before you can use a package, you must find it.

Using Packages

Packages are the preferred method to install OpenBSD software. Packages are built by the OpenBSD Project's ports team, and are expected to work without any special tweaks from the user. You must configure the software, of course, but the software itself should work as expected. Unless you are planning to make modifications to a specific piece of software, you'll be a lot happier simply installing the package fetched from a nearby mirror rather than building it from the port (or, worse, installing from the source code without the port).

Package Files and $PKG_PATH

Every package is available as a single file named after the port it is found in, a version number, and a *.tgz* extension. For example, version 2.4.2 of the `adsuck` software is available in the file *adsuck-2.4.2.tgz*.

Before you can install packages, you need to find a source for them. Find package files on the official release CDs or on OpenBSD mirror sites.

The packages are on the FTP and HTTP mirrors in the directory */pub/OpenBSD/*release*/packages/*platform. For example, the packages for the amd64 platform for OpenBSD 5.3 are in the directory */pub/OpenBSD/5.3/packages/amd64*. Look at the OpenBSD mirror list. Choose a mirror server near you, and verify that it actually has the *packages* directory for the release and platform you run. My closest mirror is *http://ftp10.usa.openbsd.org*.[2] I find the 5.3 amd64 packages at *http://ftp10.usa.openbsd.org/pub/OpenBSD/5.3/packages/amd64*.

On the official CD, you'll find packages in /release/platform*/packages*. (Downloaded installation CDs do not include packages.) If you mounted the 5.3 CD at */mnt*, you would find the packages at */mnt/5.3/amd64/packages*.

Once you've chosen a package repository, set the `$PKG_PATH` variable in your shell to it. This tells OpenBSD's package management tools where to get the packages and gives you quick access to a single authoritative source of packages.

If you set `$PKG_PATH` to an invalid location, `pkg_add` (the command for installing a package) won't work. Using a location with packages for a different architecture makes `pkg_add` give an error that packages are "not for the right architecture." If you choose an incorrect release, you'll see "bad major" or other library version errors. Either of these types of errors mean that your `$PKG_PATH` is wrong.

2. No, it's not. There is no *ftp10.usa.openbsd.org*. Follow the instructions. Look at the mirror list and pick a mirror that actually exists and is close to you. Never blindly copy my examples!

You can also list multiple package repositories. If the package tools don't find a desired package in the first repository, they try the next one. This lets you use a local package repository for your custom packages, and then fall back to the official OpenBSD repository if you don't have a local package. I use this when I must build a custom package for my network and want to use it across multiple machines.

Installing packages via FTP or HTTP is not quite as secure as installing them from CD. While the OpenBSD release team has verified all the packages on the CD set, an intruder could have tampered with whatever mirror you choose. These intrusions would be caught comparatively quickly, but it's possible that you could install packages between the time of the intrusion and the time the damage is reversed. If you're deeply concerned about package integrity, get an official CD set.

Finding Packages

As I write this, the latest OpenBSD/i386 snapshot has 7485 packages on the FTP site. This is a long list to browse through to find the specific package you want. If you have the ports tree installed, you can search it for packages, but if you wanted to use the ports tree, you wouldn't be using packages, now would you?

Say you need a piece of software that runs only on Apache 2.2. How can you find this? Find packages on the command line, or use a website.

NOTE *Most people don't need an external web server on OpenBSD; the web servers included with OpenBSD are perfectly fine for average users. I would install Apache 2.2 only if I had a specific application written for Apache 2.2. If you want to run, say, a PHP web application, just use OpenBSD's included* `nginx` *web server.*

Finding Packages on the Command Line

`pkg_info(1)` displays information about packages. While you would normally use `pkg_info` to explore the packages you've already installed, you can use `-Q` to run a case-insensitive search on the packages in your package repository. If you know part of the package name, try the package search.

```
$ pkg_info -Q apache
apache-ant-1.8.2p3
apache-couchdb-1.0.1p2
apache-httpd-2.2.22
apachetop-0.12.6
modsecurity-apache-1.9.3p5
p5-Apache-ASP-2.61p0
...
```

From the names, you can guess that the package `apache-httpd-2.2.22` contains Apache 2.2.

Finding Packages on the Web

The easiest way to search packages is to use the unofficial OpenBSD Ports website at *http://www.openports.se/*. While this isn't an official OpenBSD website, it has provided a good interface into the OpenBSD ports tree for several years. If I search for Apache on this site, the third hit is for "www/apache-httpd, apache HTTP server."

Once you know the name of the package containing the software you want, you can install it.

Installing Packages

Use `pkg_add(1)` to install packages. You don't need the version number—just the package name. Here, I install the Apache package I found earlier:

```
# pkg_add apache-httpd
❶ apache-httpd-2.2.22:libiconv-1.14: ok
apache-httpd-2.2.22:pcre-8.30: ok
...
❷ apache-httpd-2.2.22: ok
❸ The following new rcscripts were installed: /etc/rc.d/httpd2
See rc.d(8) for details.
❹ --- +apache-httpd-2.2.22 -------------------
This is the official httpd distributed by the Apache Server Project,
provided as a port for those who, for various reasons, need to run
version 2.

OpenBSD provides a custom Apache server, httpd(8), in the base system
which has been audited for security and may run in a chroot(2)
environment. Users are STRONGLY encouraged to use the system httpd
rather than this port.
```

A great deal of software requires other software to run, and OpenBSD's package tools track these *dependencies*. `pkg_add` starts my Apache installation by installing the various dependencies of the chosen package, as shown at ❶. Apache 2.2.22 requires `libiconv` and `pcre`, among several other packages. As each package installs, you'll see a progress bar scroll across the screen. If a dependency cannot be installed, the package installation terminates.

After installing all the dependencies, `pkg_add` installs the actual Apache 2.2 package, as shown at ❷. At the end of package installation, you'll see notices for startup scripts added by the package, as shown at ❸, and then any notes from the OpenBSD team about the packages, like those at ❹.

Which Files Are Installed?

Use the `-L` option to `pkg_info` to see which files a package installs.

```
$ pkg_info -L apache-httpd
Information for inst:apache-httpd-2.2.22
```

```
Files:
/usr/local/include/apache2/ap_compat.h
/usr/local/include/apache2/ap_config.h
/usr/local/include/apache2/ap_config_auto.h
/usr/local/include/apache2/ap_config_layout.h
/usr/local/include/apache2/ap_listen.h
...
```

As you can see, all of these files are installed under */usr/local*. OpenBSD installs all packages under */usr/local*.

Verbose Installation

If you're interested in the details of how `pkg_add` works, use the `-v` flag to trigger verbose mode. You can specify multiple `-v` flags for added detail. I recommend trying verbose mode a few times, in varying levels of detail, to get a deeper understanding of what `pkg_add` actually does.

Ambiguous Packages

Sometimes `pkg_add` needs an extra hint about what you want to install. For example, everything in my production network is tied together with LDAP, and I need to run an OpenLDAP mirror in each datacenter. (I could use OpenBSD's integrated LDAP daemon instead, but the master servers run OpenLDAP, and I don't want to mix LDAP servers.) The following is my attempt to install OpenLDAP.

```
# pkg_add openldap-server
❶ Ambiguous: choose package for openldap-server
 a       0: <None>
         1: openldap-server-2.3.43p10
         2: openldap-server-2.4.31p0
Your choice: 2
❷ Ambiguous: choose dependency for openldap-server-2.4.31p0:
 a       0: cyrus-sasl-2.1.25p3
         1: cyrus-sasl-2.1.25p3-db4
         2: cyrus-sasl-2.1.25p3-ldap
         3: cyrus-sasl-2.1.25p3-mysql
         4: cyrus-sasl-2.1.25p3-pgsql
         5: cyrus-sasl-2.1.25p3-sqlite3
Your choice: 2
❸ Detected loop, merging sets ok
| cyrus-sasl-2.1.25p3-ldap
| openldap-client-2.4.31
openldap-server-2.4.31p0:cyrus-sasl-2.1.25p3-ldap+openldap-client-2.4.31: ok
openldap-server-2.4.31p0:db-4.6.21v0: ok
openldap-server-2.4.31p0:icu4c-49.1.2p1: ok
openldap-server-2.4.31p0: ok
The following new rcscripts were installed: /etc/rc.d/saslauthd /etc/rc.d/
slapd
See rc.d(8) for details.
```

As you can see at ❶, OpenBSD has two OpenLDAP server packages: recent releases of version 2.3 and version 2.4. I want version 2.4. The OpenBSD OpenLDAP package is compiled with Cyrus SASL (Simple Authentication and Security Layer), which in turn comes in six different flavors, as you can see at ❷—one for each supported database. I choose the version that uses LDAP as its backend. (I don't need this particular SASL; any SASL will suffice.)

`pkg_add` realizes that this is something of a chicken-and-egg problem. LDAP is compiled using Cyrus, but Cyrus is compiled using LDAP. Fortunately, as you can see at ❸, it knows that this is a permissible configuration. The dependencies are installed, and then the OpenLDAP server that I want is added.

Identifying Where Files Originate

As you've seen in earlier examples, many packages install other packages as dependencies. Once you've installed a few complicated software packages, */usr/local* starts to fill up with weird-looking files and programs. Eventually, you'll wonder which packages are needed or where a package was installed from.

OpenBSD maintains records for every installed package in */var/db/pkg*, including files installed and dependency information, but wading through these files resembles effort, and I won't do it. Also, many package names are obscure, opaque, obfuscated, or otherwise obtuse. (It's not that the OpenBSD packages team tries to make package names incomprehensible, but there's only so much it can do when the software has a name like `icu4c`.)

Thankfully, `pkg_info(1)` can easily answer most questions about your installed software. Start by getting a complete list of all software packages on the machine with the `-a` argument.

```
$ pkg_info -a
cyrus-sasl-2.1.25p3-ldap RFC 2222 SASL (Simple Authentication and Security Layer)
db-4.6.21v0          Berkeley DB package, revision 4
icu4c-49.1.2p1       International Components for Unicode
openldap-client-2.4.31 Open source LDAP software (client)
openldap-server-2.4.31p0 Open source LDAP software (server)
quirks-1.73          exceptions to pkg_add rules
tcsh-6.18.01         extended C-shell with many useful features
```

Hang on a minute! I've installed `tcsh`, of course, as my aged brain isn't up to learning a new shell. I installed OpenLDAP, and chose to add `cyrus-SASL` as a dependency. Did `pkg_add` really install all of these other packages as dependencies? Or has one of my junior admins installed extra cruft? Do I really *need* all of these packages, or do I just need to smack a minion?

OpenBSD records which software packages you've installed, versus those installed as dependencies. Use the `-m` flag to see only those packages you manually installed.

```
# pkg_info -m
openldap-server-2.4.31p0 Open source LDAP software (server)
quirks-1.73         exceptions to pkg_add rules
tcsh-6.18.01        extended C-shell with many useful features
```

This looks more familiar. Apparently everything else really is a dependency.

Now let's look at some options. For longer descriptions of each package, add the `-d` flag or use the `-a` flag to show information for all packages. If you want to run `pkg_info` for a single package, use the package name as an argument. For example, `-L` shows the list of files a package installs. With the `-a` flag, it will show all files included in all installed packages, but that's probably more than you want. To show all files installed by a package, use the `-L` flag and the package name.

```
$ pkg_info -L tcsh
Information for inst:tcsh-6.18.01

Files:
/usr/local/bin/tcsh
/usr/local/man/man1/tcsh.1
/usr/local/share/nls/C/tcsh.cat
/usr/local/share/nls/de_AT.ISO_8859-1/tcsh.cat
/usr/local/share/nls/de_CH.ISO_8859-1/tcsh.cat
/usr/local/share/nls/de_DE.ISO_8859-1/tcsh.cat
...
```

As you can see, the `tcsh(1)` package includes the actual `tcsh` binary, the man page, and a whole bunch of National Language Support (NLS) files. Given a package name, you can identify which files are part of the package.

Going the other way, sometimes you want to know where a particular file originated. For example, I occasionally browse my server filesystems looking for weird stuff. I define "weird stuff" as "things I don't recognize." If I see an unfamiliar program or file, I'll check to see which package installed it.

```
$ pkg_info -E /usr/local/sbin/pluginviewer
/usr/local/sbin/pluginviewer: cyrus-sasl-2.1.25p3-ldap
cyrus-sasl-2.1.25p3-ldap RFC 2222 SASL (Simple Authentication and Security
Layer)
```

The only `pluginviewer` I had previously encountered was one designed to help Unix web browsers run third-party software when a website demanded a plug-in. I don't know what this `pluginviewer` does, but apparently it's a legitimate part of `cyrus-SASL`. To find something to worry about, I need to keep looking.[3] If you do many file searches like this, you can get faster results by using `pkglocatedb` (*/usr/ports/databases/pkglocatedb*).

3. If you don't see anything to worry about on any given server, you aren't looking hard enough.

After installation, many packages show a message, which I frequently read and promptly forget. To display this information again, use `pkg_info` with the `-M` flag.

```
$ pkg_info -M apache-httpd
Information for inst:apache-httpd-2.2.22

Install notice:
This is the official httpd distributed by the Apache Server Project,
...
```

If you don't remember which package had the message you wanted, use the `-a` flag instead of a package name to display the messages for all packages that have one. To show all packages that are not required by other packages, use the `-t` flag, which you might think matches all packages you chose to install. If you didn't request a package, it could only be installed as a dependency to something you requested, right?

```
$ pkg_info -t
apache-httpd-2.2.22 apache HTTP server
icu4c-49.1.2p1      International Components for Unicode
quirks-1.73         exceptions to pkg_add rules
tcsh-6.18.01        extended C-shell with many useful features
```

I know that I did not choose to install `icu4c`. I have no moral objections to the software, mind you, but it's nothing I requested. How did a piece of software that I didn't choose to install, and isn't required by anything else, get on this system?

It's there because I uninstalled something that required it.

Uninstalling Packages

To remove a previously installed package, use `pkg_delete(1)`.

```
# pkg_delete openldap-server
openldap-server-2.4.31p0: ok
Read shared items: ok
--- -openldap-server-2.4.31p0 -------------------
You should also run /usr/sbin/userdel _openldap
You should also run /usr/sbin/groupdel _openldap
```

`pkg_delete` does not request confirmation. It doesn't ask if you are sure. It just blasts the software off the disk and gets on with its day. It also doesn't remove the unprivileged users and groups created for the software, as you might still have files owned by them.

Remember that many packages require other packages. By default, `pkg_delete` doesn't remove dependencies of packages you remove. For example, earlier we saw that `icu4c` had been installed automatically as a leftover dependency from a removed OpenLDAP server package. To automatically

remove unneeded dependencies, use the `-a` flag. For example, to completely eradicate the `openldap-server` package and its infrastructure from the machine, run `pkg_delete` twice.

```
# pkg_delete openldap-server
# pkg_delete -a
```

This should clean your system of all packages installed as dependencies.

Package Limitations

The package system is fast, efficient, reliable, and the OpenBSD Project's preferred way for users to install software. But the system does have a few limitations that you should be aware of, including lags in the software-porting process and the support for newer packages on older versions of OpenBSD.

Each OpenBSD release supports only packages built for that release, and new packages are not built for old releases. The packages issued with the release are all you'll get. (There are slight exceptions to this if you're running `-stable`; see Chapter 20.) If you're running OpenBSD 5.3 and try to install packages from OpenBSD 5.4, they won't work.

Most packages include software produced by third parties. OpenBSD provides the packaging, but the software itself is released on a schedule completely independent of OpenBSD's. After the software developers release their newest software, the OpenBSD package is updated, but there's a gap between the software's release date and the release of the OpenBSD package. A popular package might be updated in hours, while larger, less frequently used, or unpopular packages can languish at an older version for days or weeks. These packages are not officially available until the next OpenBSD release, so you might run software that's a point or two behind the latest for a few months. Usually, this is not a problem (if it is, investigate OpenBSD's `-stable` branch, discussed in Chapter 20.)

NOTE *If packages won't work for you, investigate building third-party software through ports. You won't get newer versions of the software, but you can get slightly different versions.*

Using Ports

The ports collection is the toolkit to build OpenBSD packages. Installing software from ports takes longer than installing via packages, is more error-prone, and requires a deeper understanding of the system and the add-on software than packages demand. You can't get packages for every possible situation, however (one particularly annoying example is when the license for a particular piece of software makes it illegal for the OpenBSD project to create and distribute packages), and sometimes ports are the only way to get third-party software on your OpenBSD system short of compiling it yourself.

What makes ports interesting is their level of automation. With one command, a port can find the source code for a program, download it, verify its integrity, apply any patches needed to make it run on OpenBSD, toggle any flags needed for any custom features of your system, build the code into actual binaries, produce a package, and install it. If you have compiled software on other platforms, you'll quickly realize how ports simplify building software.

Like packages, ports work only on the version of OpenBSD for which they are released. That means that you must use the OpenBSD 5.4 ports collection on OpenBSD 5.4; the 5.5 ports collection won't work. Oh, it might look like it works sometimes, but the software will fail unpredictably, and no one will have sympathy for you (sympathy for your coworkers, perhaps, but not for you).

When you upgrade OpenBSD, the expectation is that you will upgrade your ports collection and all installed packages to the precise matching version. You might be able to use older packages on a newer OpenBSD, as long as you don't delete the older shared libraries required by the software.

The Ports Tree

The ports tree is usually installed in */usr/ports*. If you want the ports tree, you must manually fetch the *ports.tar.gz* file from your OpenBSD release and extract it under */usr*.

NOTE *I suggested this way back in Chapter 4, but you can also get the ports tree and keep the files up to date using* `cvs(1)`*, as covered in Chapter 20. Look in this directory, and you'll find a whole bunch of directories and files.*

The *INDEX* file contains a list of every port in the system, in alphabetical order but machine-readable format. You can search this file for ports, but I recommend using one of the tools discussed later to do so.

The *Makefile* contains the basic machine instructions for making the ports system work. While it's intended for use by `make(1)`, you can learn a lot by reading the makefile for any port. Most of the really complicated ports code is in the *ports/infrastructure* directory, and all of the makefiles in the ports system build on that infrastructure.

The remaining directories are software categories. Each category contains a further layer of directories, and each directory under a category is a port of a specific piece of software. OpenBSD has more than 7600 ports as of this writing, so this hierarchical organization is vital to keeping them in some sort of manageable order.

For example, the following is a listing of the contents of the *news* directory, which contains programs for using and managing Usenet news. This is one of the smaller categories. Some categories have hundreds of entries, but they're arranged in much the same way.

```
CVS             leafnode        p5-News-Article  py-yenc         tin
Makefile        newsfetch       p5-News-Newsrc   sabnzbd         trn
aub             nn              pan              sickbeard       ubh
hellanzb        p5-Gateway      plor             slrn
yencode
```

Like the *CVS* directory in the main ports tree, the category's *CVS* directory contains CVS version control information that doesn't matter for day-to-day operation. The *Makefile* contains a list of valid ports within the category. You can build all of the ports in this category using this makefile, although that's mostly useful only when building packages en masse. (When the OpenBSD Project team builds everything in the ports tree, it uses */usr/ports/infrastructure/bin/dpb*.)

Let's go down another level. Here's the port for `tcsh`, one of my non-negotiable requirements as a sysadmin:

```
$ ls /usr/ports/shells/tcsh
CVS      Makefile distinfo patches  pkg
```

The *CVS* directory contains version control information, as in every *CVS* directory.

The *Makefile* gives specific instructions for building `tcsh` on OpenBSD, including where to get the software and any patches, how to extract it, where the package can be distributed from, and any supported customizations.

The *distinfo* file contains several different cryptographic hashes for the source code to be downloaded, to avoid building software from compromised source code, and the size of the source file. Newer ports contain only SHA-256 hashes.

NOTE *While it's possible (difficult, but possible) to have a compromised file match a specific hash, it's extremely unlikely that an altered source code file could match hashes computed with several different algorithms and have the same size as the uncompromised code. Even if people figure out how to break a particular hash, use of multiple hashes and the file size make compromising a source file nearly impossible.*

The *patches* directory contains code alterations needed to make this software run on OpenBSD. Some ports have no patches; others have dozens.

Finally, the *pkg* directory describes the package and lists the files that the complete package must include.

Secondary Ports

Some ports include other ports. Here are the contents of the *emulators/fedora* port.

```
CVS    Makefile   Makefile.inc  base       cups       motif      sdl
```

The file *Makefile.inc* is new, as are the subdirectories *base*, *cups*, *motif*, and *sdl*. The subdirectories are independent ports. These four ports are often installed together, and as a whole, support OpenBSD's Linux emulation (documented in `compat_linux(8)`). All four ports call in the common instructions in *Makefile.inc*. (The ports tree doesn't include many of these, but don't be shocked when you find one.)

Read-Only Ports Tree

The process of building a port creates an installable package and uses a whole bunch of temporary files, source files, and status files. By default, all of these files are placed inside the ports tree itself. While this works, I encourage you to treat */usr/ports* as a read-only OpenBSD directory tree, just like */usr/bin*, */usr/lib*, and so on. Doing so simplifies upgrading and identifying local changes, helps identify what you've built from ports, and saves space on the */usr* partition.

NOTE *Build files for ports can range from a few kilobytes to several gigabytes, so it's best to build ports on a large scratch partition. If you have unpartitioned disk space, create a partition just for building ports. Or use any partition with space, or even an NFS partition.*

Configure the ports collection by setting variables in */etc/mk.conf*. To use a read-only ports tree, set the variables in these directories:

`WRKOBJDIR` Directory where the software is extracted from source and compiled. These can be deleted and re-created as needed.

`PACKAGE_REPOSITORY` Directory where completed packages are stored. The ports collection builds packages, which you can then install.

`PLIST_DB` Directory where package packing lists are stored.

`BULK_COOKIES_DIR` Directory for storing status cookies during mass builds of packages.

`UPDATE_COOKIES_DIR` Directory for storing status cookies during mass updates of packages.

`DISTDIR` Directory where vendor source code is kept. Source code is usually retained for reuse.

If these directories are owned by your regular user account, you can do a large part of package building without being root.

On one particular test system, I have hundreds of gigabytes free in */home*, so I chose to put my package directories there. Here's my */etc/mk.conf*:

```
WRKOBJDIR=/home/ports/wrkobjdir
DISTDIR=/home/ports/distdir
PLIST_DB=/home/ports/plist
BULK_COOKIES_DIR=/home/ports/bulk_cookies
UPDATE_COOKIES_DIR=/home/ports/update_cookies
PACKAGE_REPOSITORY=/home/ports/pkgrepo
```

The ports system will build everything in */home/ports/wrkobjdir*. Original source code files go in */home/ports/distdir*. The ports system maintains various records in */home/ports/update_cookies* and */home/ports/bulk_cookies*. Completed packages go into */home/ports/pkgrepo*.

NOTE *If you have a dedicated port-building machine, consider per-release package repositories. For example, I might have three versions of OpenBSD running at any given time. The package-building machine always runs the latest release, but I don't want to throw away my old packages, so I use a package repository directory like* /home/ports/pkgrepo/5.4 *for packages built on a 5.4 system.*

Finding Software

As with packages, the first problem with ports is finding software you want. (To randomly poke around the ports tree in a pretty interface, see the *http://www.openports.se* website.) OpenBSD has several ways to search the ports collection, including the ports index, keywords, and via SQL.

The Ports Index

The file */usr/ports/INDEX* lists all software in the ports tree, sorted by category and then alphabetically. If you have a good idea what your port is called, you can search the file for your preferred software. The index describes each port in a single pipe-delimited line, much like this:

```
gcpio-2.11|archivers/gcpio||GNU copy-in/out (cpio)|archivers/gcpio/pkg/
DESCR|The OpenBSD ports mailing-list <ports@openbsd.org>|archivers|
STEM->=0.10.38:devel/gettext converters/libiconv|STEM->=0.10.38:devel/
gettext|STEM->=0.10.38:devel/gettext|any|y|y|y
```

While the ports tree itself finds this a convenient format, it's not particularly human-readable. To translate this to a human-friendly format, go into */usr/ports* and run **`make print-index`**. (This process goes on for tens of thousands of lines, so feed it to a pager.) Here's the same port in the human-readable format:

```
$ cd /usr/ports
$ make print-index | less
...
Port:   gcpio-2.11
Path:   archivers/gcpio
Info:   GNU copy-in/out (cpio)
Maint:  The OpenBSD ports mailing-list <ports@openbsd.org>
Index:  archivers
L-deps: STEM->=0.10.38:devel/gettext converters/libiconv
B-deps: STEM->=0.10.38:devel/gettext
R-deps: STEM->=0.10.38:devel/gettext
...
```

The Port statement gives the official name of the port and the version of the ported software. This software is called gcpio, and it's at version 2.11. The Path gives the ports tree category and directory where the port can be found—in this case, *archivers/gcpio*. The Info line gives a very brief description of the software. This is the GNU version of cpio(1). The Maint, or maintainer, is the person or group responsible for maintaining this software in the ports tree. The OpenBSD ports team supports the gcpio port. The best-maintained ports usually have an individual as a maintainer, rather than the mailing list.

The final three entries describe other software required by this software. The L-deps line lists shared libraries, B-deps lists software needed to build this port, and R-deps lists the port's runtime dependencies.

What good does this do? Suppose you're still hung up on an Apache 2 web server. You can search *INDEX* for ports beginning with "apache."

```
$ grep -i ^apache INDEX
...
apache-httpd-2.2.20p1|www/apache-httpd||apache HTTP server|www/apache-httpd/
pkg/DESCR|The OpenBSD ports mailing-list <ports@openbsd.org>|www net|
apr-util-*-!ldap:devel/apr-util converters/libiconv devel/pcre|STEM->=1.21:
textproc/groff|converters/libiconv|any|y|y|y|y
```

The first three (omitted) entries are ports related to Apache, but they are not the web server software. The fourth line is our port.

Gathering this information from the index is rather limited, however. If you don't know the name of the software, or how OpenBSD packages the software, you can't easily find the port. In that case, try one of the other methods discussed next.

Finding by Keyword

If you don't know a package's exact name, try the ports collection's search feature: make search and a key scans the index for a specific word. To search for Apache-related software, try this:

```
$ make search key=apache
```

On my system, this returns 62 results. You'll need to scroll through several pages of possibilities, but you'll find what you want.

You might need to try several possible keywords for a particular package, as some keywords have no hits and others generate too many.

Finding via SQL

The sqlports package lets you build a database of the *INDEX* file, permitting you to search for ports based on highly arbitrary criteria via SQL. For example, say you want to know all ports that depend on libiconv and expat. In this case, sqlports is your friend. Install it from ports or packages, and it will automatically build a database in */usr/local/share/sqlports* from *INDEX*, and then use OpenBSD's sqlite3 to query the database.

I won't teach SQL[4] here, but just as an example, here's how to search for ports whose name includes the string "apache" using `sqlports` (which can build much more complex queries than this one):

```
$ sqlite3 /usr/local/share/sqlports
sqlite> select fullpkgname from ports where fullpkgname like '%apache%';
apache-couchdb-1.0.1p2
apache-ant-1.8.2p3
apachetop-0.12.6
apache-httpd-2.2.22
modsecurity-apache-1.9.3p5
p5-Apache-ASP-2.61p0
p5-Apache-DB-0.14p3
...
```

The Apache `httpd` server is the fourth hit, but there are another dozen or so ports. Every name that begins with `p5-` is a Perl module.

Building Ports

You've decided to ignore the OpenBSD team's recommendations to use packages, downloaded and extracted the ports tree, found software you need to install from ports, and designated an area for building ports. Now what?

The port directories don't contain actual source code. When you build a package from a port, the system does the following:

- Automatically downloads the appropriate source code from an approved Internet site
- Checks the downloaded code for integrity errors
- Extracts the code to the build area
- Patches the code
- Compiles the code
- Creates the package
- Installs the package (optional)

Additionally, if the port you're adding has unmet dependencies, the system also handles installing those dependencies.

To make all this happen, just go to the *port* directory and enter this command:

```
# make install
```

You should see the port build the software, create the package, and install the package on your system.

4. This example exhausts my understanding of SQL. As long as I maintain my database ignorance, people won't expect my help fixing their databases.

What a Port Installation Does

It's time to dissect a port build and installation. Here's how to install `tcsh` from a port:

```
# cd /usr/ports/shells/tcsh
# make install
===>  Verifying specs: c termlib c termlib
===>  found c.65.0 termlib.12.1
===>  Checking files for tcsh-6.18.01
>> Fetch ftp://ftp.astron.com/pub/tcsh/tcsh-6.18.01.tar.gz
tcsh-6.18.01.tar.gz 100% |**************************************************
|   905 KB    00:00
>> (SHA256) tcsh-6.18.01.tar.gz: OK
```

The port first checks to see if the software's required libraries are in place. Building `tcsh` requires the `termlib` and `c` libraries. The port finds `termlib` but not a file containing the `tcsh` source code on the local system, so the port fetches the code. (When building a port, you should see the system downloading the appropriate source code.) The port then verifies the checksum of the downloaded code. If the port can't get all of the code, or the checksums don't match, the build process stops.

Once all necessary source code is downloaded and verified, the build continues with something like this:

```
...
===>  Extracting for tcsh-6.18.01
===>  Patching for tcsh-6.18.01
===>  Configuring for tcsh-6.18.01
Using /usr/ports/pobj/tcsh-6.18.01/config.site (generated)
configure: WARNING: unrecognized options: --disable-silent-rules
configure: loading site script /usr/ports/pobj/tcsh-6.18.01/config.site
checking for a BSD-compatible install... /usr/bin/install -c -o root -g bin
checking build system type... i386-unknown-openbsd5.2
checking host system type... i386-unknown-openbsd5.2
...
```

The port extracts the source code from the compressed file(s), applies any OpenBSD-specific patches, and starts the build process. (Many of you know that `configure` is not the same as building software, but not all software requires a `configure` step. The port knows what to do.)

The build process will go on for many lines. Building something like OpenOffice can take days and generate hundreds of thousands of lines of output.

NOTE *If you need to debug a port build failure, those messages that scroll off the top of your screen or terminal window contain all the clues you get. For that reason, I often build ports in a `script(1)` session. If you like the idea of keeping build messages around, see the `script` man page for details.*

Eventually, you should see a message that the build has finished and the port is installing the software.

```
...
===>  Faking installation for tcsh-6.18.01
install -c -s -o root -g bin -m 555 /home/ports/wrkobjdir/tcsh-6.18.01/tcsh-
6.18.01/tcsh /home/ports/wrkobjdir/tcsh-6.18.01/fake-i386/usr/local/bin/tcsh
install -c -o root -g bin -m 444 /home/ports/wrkobjdir/tcsh-6.18.01/tcsh-
6.18.01/tcsh.man /home/ports/wrkobjdir/tcsh-6.18.01/fake-i386/usr/local/man/
man1/tcsh.1
install -c -o root -g bin -m 444 /home/ports/wrkobjdir/tcsh-6.18.01/tcsh-
6.18.01/nls/C.cat /home/ports/wrkobjdir/tcsh-6.18.01/fake-i386/usr/local/
share/nls/C/tcsh.cat
...
```

The port installs the software in a temporary location in the port building directory, but that's not where we want the software installed! Remember that the ports system builds packages, and then installs from the package. This "fake" installation is for building the package.

```
...
===>  Building package for tcsh-6.18.01
Create /home/ports/pkgrepo/i386/all/tcsh-6.18.01.tgz
...
```

There's the package, retained in the package repository specified earlier. You might want to grab this file to install on your other machines, or perhaps even share the package repository via NFS.

Now, because we specified `make install` on the command line, the port installs the created package.

```
...
===>  Verifying specs: c termlib
===>  found c.65.0 termlib.12.1
===>  Installing tcsh-6.18.01 from /home/ports/pkgrepo/i386/all/
...
tcsh-6.18.01: ok
#
```

Installing the package requires making some of the same checks as building the package. Yes, the port could not have built the package without those libraries, but the ports system doesn't assume that the package was built on the local system.

Port Build Stages

The package build process actually includes several stages, or smaller chunks of build procedure. Each stage performs all the stages before it. The final stage, `make install`, calls all of them, which provides several points where you can intervene in the port build process. If you want to make custom changes to a package, you can do it here.

Let's look at each of the stages called for every port build.

The make fetch Stage

The `make fetch` stage gets the source code, or *distfiles*, for the port. First, it looks in any directories specified by the *mk.conf* variable `$DISTDIR`. If this variable isn't set, it looks in the directory specified by the shell environment variable `$DISTDIR`. If neither variable is set, it looks in */usr/ports/distfiles*. If `make fetch` finds the distribution files and thinks that they're the correct version, it hands control to the next requested stage, and the build continues.

If the source code is not on the local machine, `make fetch` tries to download it from an Internet site specified in the port's makefile as `MASTER_SITES`. (You can customize download locations, as discussed in "Customizing Ports" on page 246.)

You'll find the `make fetch` command very useful when there are certain times in your day when you can download more easily than other times. For example, I have a T1 to my house,[5] but my employer's office has roughly 66 times as much bandwidth as I have at home. I can run `make fetch` on my laptop while visiting my employer, go home, and build the port in peace. (And the boss thinks I come in because he buys lunch.)

The make checksum Stage

The `make checksum` stage verifies that distfiles have not been corrupted, either by the download process or maliciously. OpenBSD includes several different checksums for each distfile, but only checks that the SHA-256 checksum matches the distfile. If the checksum matches, the build proceeds to the next stage. If the checksums do not match, the build immediately aborts. The build will not continue until you find a distfile that matches the checksum.

Not all software developers are conscientious about updating the names of their distfiles when they update their software. For these software packages, the *foo-1.0.tgz* file the port developer downloaded in the morning might differ from the *foo-1.0.tgz* file you download later that same day. Perhaps the original software author thought that no one would notice, but the OpenBSD folks would, if only via the logic built into the ports tools. After all, the ports system can't tell the difference between a source file quietly modified by the software author and a source file quietly modified by an intruder. If you get a distfile that doesn't match the recorded checksum, try to fetch a matching file by setting the `REFETCH` variable to `true`.

```
# make checksum REFETCH=true
```

Now `make` will walk through all the distfile sources listed in the port, downloading them successively in an effort to find a distfile that matches that used by the port developer.

If you are absolutely certain that the file you downloaded is the correct, untampered-with one, but it still doesn't pass `make checksum`, you're wrong. If you know that you're wrong, but you really do want to install compromised

5. Don't laugh. It's paid for.

or damaged software, set the environment variable `NO_CHECKSUM=yes` to skip the `make checksum` stage.

WARNING *Skipping the `make checksum` stage might be valid for debugging, but it certainly isn't the way to create a stable, useful, or secure package. You also might invalidate the rest of the port. Perhaps the OpenBSD patches will no longer apply cleanly, the software just won't run, or you could even be installing a backdoor, inviting scumbags to stash problematic content on your machine. You are utterly on your own if you insist on ignoring a checksum mismatch.*

The make prepare Stage

At this point, the ports system gets into recursion. At `make prepare`, the port checks for any software needed to build or run the software you're trying to build. If the port lists any of these dependencies, it checks to see if they are installed. If the dependencies are not installed, this stage kicks off `make install` for those required ports. Once all of the required dependencies are installed, this stage ends.

The make extract Stage

The ports system must extract the source code from the distfile before building the software. Source code is extracted into the directory defined by `$WRKOBJDIR`, or in a directory under */usr/ports/pobj/* named after the port. By default, my `tcsh` port would extract under */usr/ports/pobj/tcsh*, but because I defined a separate location for building software, it's built under */home/ports/wrkobjdir/tcsh*.

The make patch Stage

Any patches included in the port's patches directory are applied in the `make patch` stage. If the patches all apply correctly, this stage ends. If the patches do not apply correctly, the port fails.

To apply your own patches to the port, or to review the code before compiling it, run `make patch` first. Your patches might conflict with the port patches if you apply them first, cause compilation failures, or bring up any number of other problems. By running `make patch` first, you get to see the code as OpenBSD can compile it. Anything you break after that is definitely your fault.

The make configure Stage

Many software packages use a `configure` script to prepare themselves for compilation on a specific platform. The `make configure` command runs that script. If you want to edit the `configure` script, do so before running this stage! If there is no `configure` script, the port silently skips this stage.

The make build Stage

The `make build` stage compiles the fetched, extracted, patched, and configured software. If you type `make` in a port directory, the port calls `make build`. This stage doesn't assemble a package; it just performs the compilation and creates the actual program binaries in the port's work directory.

The make fake Stage

The `make fake` stage installs the software in a subdirectory, laid out exactly as it would be under the *root* directory. This fake root directory is in the work directory, named *fake-* with the architecture appended, such as *fake-amd64*. Everything that will be in the package is installed under this directory, with the same ownership and permissions that it will include in the package.

The make package Stage

The `make package` stage bundles up the port's fake installation directory, adds in packaging and installation instructions, and ties it all up in a package exactly like those available on the FTP site. The package will be stored under the *PKGREPO* directory you defined earlier (or in */usr/ports/packages* if you didn't define one), in a subdirectory organized by architecture, and in further subdirectories organized by available distribution locations.

`make package` means that you can build this port on one machine without installing it. You must install the build dependencies to build the port, however.

The make install Stage

The `make install` stage runs `pkg_add(1)` to install the package you compiled.

The make clean Stage

Some packages require a lot of disk space. The `make clean` stage removes all of the build files except the distfile and the completed package.

Customizing Ports

OpenBSD includes a variety of hooks to let you easily customize how you get and build ports. If possible, you should use the OpenBSD-provided infrastructure, but there may be cases where that's not possible. Here, we'll look at some of the more commonly used customization settings.

Local Distfile Mirrors

While ports provide several places to get source code, you might want to override those sites. Perhaps you share a network with a major mirror site, or you don't have unfettered Internet access. OpenBSD lets you set your own preferred mirror sites.

Preferred Collection Mirrors

Many software sources can be grouped into *collections*, which tend to be mirrored together. An example is the official GNU software collection. A GNU mirror site probably has everything in the official GNU collection. The Gnu C Compiler Project has its own set of software and mirrors. There are older software collections, such as SunSITE, and newer ones, such as SourceForge.

Each collection is available from a list of mirror sites. OpenBSD maintains lists of these mirror sites in */usr/ports/infrastructure/templates/network.conf.template*. Never edit this file; it's a core ports file, and upgrading changes it.

For example, here's a list of mirrors for a smaller project, BerliOS:

```
...
MASTER_SITE_BERLIOS+=   \
        http://download.berlios.de/ \
        http://download2.berlios.de/ \
        http://spacehopper.org/mirrors/berlios/
...
```

Several ports want to fetch BerliOS-related software from the main BerliOS download site. The OpenBSD port developers have identified three desirable mirrors, as listed in the variable `MASTER_SITE_BERLIOS`.

But suppose you have a BerliOS mirror much closer to you. Perhaps it's not an official mirror, or you've managed to finagle access to a nonpublic mirror. It's closer, it's faster, and you would prefer to use it. OpenBSD looks at */usr/ports/infrastructure/db/network.conf* before the default mirror list. You could copy the default mirror list to this file and edit it, but then you would need to manually synchronize changes during upgrades. That's work, and therefore morally questionable. Instead, add entries only in *network.conf*, and include the default *network.conf.template*.

Suppose you have a private BerliOS mirror at *http://www.blackhelicopters.org/berlios/*. You would create a *network.conf* file like this:

```
MASTER_SITE_BERLIOS+=   \
        http://www.blackhelicopters.org/berlios/

.include "../templates/network.conf.template"
```

The `+=` used in both *network.conf* and *network.conf.template* means "Add this value to variable such-and-such." More desirable mirrors appear first in the list. This *network.conf* entry adds the private mirror to the variable `MASTER_SITE_BERLIOS`, and then calls in *network.conf.default*, which appends all of the other mirrors. The end result is that the BerliOS mirror list will contain four mirrors: your preferred mirror first and the default OpenBSD-approved mirrors later. If a file does not exist on a mirror, the port will try the other mirrors in order.

I used BerliOS as an example because it has a small mirror list, but the same applies to any other software collection that OpenBSD recognizes. Other collections available at this time are shown in Table 13-1.

Table 13-1: Some Software Collections

Collection	Description
MASTER_SITE_GNU	Software from the GNU project
MASTER_SITE_GCC	Software from the GCC project
MASTER_SITE_XCONTRIB	Contributions to the X Window System
MASTER_SITE_R5CONTRIB	Older X Window System contributions
MASTER_SITE_SUNSITE	A collection of Sun software
MASTER_SITE_SOURCEFORGE	Software hosted by SourceForge
MASTER_SITE_SOURCEFORGE_JP	Japanese SourceForge mirrors
MASTER_SITE_GNOME	Software from the Gnome project
MASTER_SITE_PERL_CPAN	The biggest Perl module collection
MASTER_SITE_TEX_CTAN	Software for TeX typesetting
MASTER_SITE_KDE	Software related to KDE
MASTER_SITE_SAVANNAH	Software development hosted by the FSF
MASTER_SITE_AFTERSTEP	Software related to the AfterStep window manager
MASTER_SITE_WINDOWMAKER	Software related to the Window Maker window manager
MASTER_SITE_FREEBSD_LOCAL	Software distributed by the FreeBSD Project, but not included in FreeBSD
MASTER_SITE_PACKETSTORM	Security software part of the Packet Storm collection
MASTER_SITE_APACHE	Apache Foundation software
MASTER_SITE_BERLIOS	Parts of the BerliOS Linux project
MASTER_SITE_MYSQL	Software from the MySQL project (Oracle)
MASTER_SITE_PYPI	Python software
MASTER_SITE_RUBYGEMS	Modules for Ruby
MASTER_SITE_NPM	JavaScript packages
MASTER_SITE_ISC	Software from the Internet Software Consortium

If you have a Debian mirror in your university datacenter, list it in *network.conf*. If it appears a second time, later in the list, because it's listed in *network.conf.template*, so what? Either the distfile is there, in which case you save time and bandwidth, or the *distfile* isn't there, in which case you waste 50 milliseconds checking the local mirror a second time.

Fallback Mirrors

OpenBSD supports two fallback mirrors. If all other distfile sources fail, you can check either the OpenBSD or FreeBSD mirrors for the file. Both OpenBSD and FreeBSD tend to mirror distfiles for active ports. This isn't

preferred, because if everyone did this, it would use bandwidth that the projects need for distributing their own software. But if you're desperate, set MASTER_SITE_OPENBSD and/or MASTER_SITE_FREEBSD to YES in *network.conf*.

Primary Mirror

You can have the ports system check a particular site first for all distfiles, regardless of the download site listed in the port. Perhaps you have a local mirror where you've stuck a whole bunch of distfiles, or you automatically load distfiles from your ports-building machines to a central location. Define this site with the variable MASTER_SITE_OVERRIDE in *network.conf*.

```
MASTER_SITE_OVERRIDE=ftp://ftp.mycompany.com/distfiles
```

NOTE *I've built local distfile mirrors many times, usually when starting a new job. I manage to update the mirror for about six months before some other task supersedes it and the mirror becomes obsolete, so I don't generally recommend this practice. But if maintaining a local distfile mirror reduces your workload instead of increasing it, enjoy.*

Flavors

Some ports can create multiple but slightly different packages through *flavors*. The Apache 2.2 web server I keep dragging out as an example can be built with or without LDAP support, as can programs with optional X support. Shells can be built in dynamic or static versions. OpenBSD's official packages are built with the most common choices, but these alternatives are reasonable and occasionally necessary.

To identify the flavors that a port supports, go to the port directory and run **make show=FLAVORS**. Here's how to check the flavors of the popular text editor Vim:

```
# cd /usr/ports/editors/vim
# make show=FLAVORS
huge gtk2 athena motif no_x11 perl python ruby
```

You can guess what some of these eight flavors do, but how can you learn about the others? You can check the package's description file for brief descriptions of each flavor. Here are the descriptions for the Vim flavors, from *editors/vim/pkg/DESCR-main*:

```
...
Flavors:
        gtk2       - build using the Gtk+2 toolkit (default);
        motif      - build using the Motif toolkit;         athena     - build
using the Athena toolkit;
...
```

Motif? I remember Motif. And now I'm going to try really hard to forget it again. But if you want Motif support in your Vim version, go for it.

To fall back to my ongoing example, here are the flavors for Apache 2:

```
# cd /usr/ports/www/apache-httpd
# make show=FLAVORS
ldap
```

I use LDAP to attach websites to my central authentication system. If I can get LDAP authentication on my web server, I want it.

Building a Flavored Port

Define any desired flavors with the `$FLAVOR` environment variable, but not in your *.profile* or *.cshrc* file, as a port will not build if you request an unrecognized flavor. Define it when you build the port. For example, while still in the *apache-httpd* directory, I run this command:

```
# env FLAVOR="ldap" make package
===>  Checking files for apache-httpd-2.2.20p1-ldap
>> Fetch http://www.reverse.net/pub/apache/httpd/httpd-2.2.20.tar.gz
...
===> apache-httpd-2.2.20p1-ldap depends on: openldap-client-* - not found
===>  Verifying install for openldap-client-* in databases/openldap
...
```

By your defining the flavor on the command line, the port knows to check for the OpenLDAP client needed to build Apache. When the build finishes, you should get a package file with the flavor appended—in this case, *apache-httpd-2.2.20p1-ldap.tgz*.

Flavors and Dependencies

When you build a *flavored port*, the flavor does not propagate to dependencies. You need to check the flavored port's dependencies to see if they need flavoring as well. For example, my flavored Apache package calls in the OpenLDAP client, which has no flavors, but OpenLDAP calls in `cyrus-SASL`, and if I check that port, I see this:

```
# cd /usr/ports/security/cyrus-sasl2
# make show=FLAVORS
db4 ldap mysql pgsql sqlite3
```

Cyrus SASL comes in LDAP flavor, but defining that I want Apache built in LDAP flavor doesn't mean that `cyrus-SASL` will also be built with LDAP support. If I need LDAP support in this dependency, I must build it separately. I don't need it for my environment, so I won't bother, but check for potential issues like these when building your packages.

If you decide to rebuild a dependent port with a flavor, be sure to rebuild all the ports that depend on that port afterward. Be sure that your packages have correct dependencies using the targets `print-build-depends` and `print-run-depends`. Here, I see which ports I'll need to build for my flavored Apache 2:

```
# env FLAVOR="ldap" make print-build-depends
This port requires package(s) "metaauto-1.0 gperf-3.0.4 libiconv-1.14
gettext-0.18.1p1 gmake-3.82p1 groff-1.21p8 pcre-8.30 help2man-1.29p0
autoconf-2.65 autoconf-2.68 cyrus-sasl-2.1.25p3 icu4c-4.8.1.1p0 db-4.6.21v0
openldap-client-2.4.31 apr-1.4.6 apr-util-1.4.1-ldap" to build.
```

I can check the flavors of each of these ports.

Building Multiple Flavors

You can build multiple flavors of one port on the same system. Each package filename includes the flavor, so you can have packages for both the Motif and GTK2 versions of Vim. Carefully inspect the dependencies to verify that each is built with the correct flavoring. For packages with flavored dependencies, I recommend removing every flavored dependency and rebuilding them all again, so that everything gets the correct flavor.

Uninstalling and Reinstalling Flavored Ports

Flavoring a package changes its name. I can't run `pkg_delete apache-httpd` because it's not installed. Query the system for the packages you've manually installed, and you'll see this:

```
# pkg_info -m
apache-httpd-2.2.20p1-ldap apache HTTP server
...
```

When working with this package, you must specify the flavor.

```
# pkg_delete apache-httpd-2.2.20p1-ldap
apache-httpd-2.2.20p1-ldap: ok
...
```

Similarly, to reinstall a flavored package, specify the flavored package file.

Subpackages

Some ports contain multiple wildly different packages. This isn't like adding LDAP support to Apache or Motif support to Vim—those are changes to the existing package, not wildly different. Some ports create two completely different packages, such as a database client and the associated

database server. I've drawn in OpenLDAP through this chapter's examples, and both the OpenLDAP server and client come from the same port: *databases/openldap*. Other applications might have plug-ins for accessing several different database engines. These are called *subpackages* or *multipackages*.

Unlike flavors, OpenBSD provides all subpackages of a port. You can install both the server and client versions of OpenLDAP from official packages. When the port is built, all the subpackages are built. The package is split into subpackages at the package-bundling stage.

To see all the subpackages supported by a port, run the following command:

```
# cd /usr/ports/databases/openldap
# make show=MULTI_PACKAGES
-main -server
```

This port has two subpackages: `openldap-main` and `openldap-server`.

How can you learn what each subpackage includes? As with flavors, you can check its description file, which is *pkg/DESCR*. OpenLDAP includes *pkg/DESCR-server* and *pkg/DESCR-main*. Reading these shows that the `main` package is the client, as you would expect.

If you run `make install` in the port directory, you get the main version of the port—in this case, the OpenLDAP client. OpenLDAP clients outnumber the servers, so that's also what you would expect. To build a different subpackage, set `SUBPACKAGE` in the environment on the command line, as you did for flavors.

```
# env SUBPACKAGE="-server" make package
```

This builds the `-server` version. Be sure to include the leading dash, as specifying a nonexistent subpackage makes the build fail.

Packages and rc.d Scripts

Chapter 5 covered how to have OpenBSD start packaged software, but let's review it quickly. When you install a package that can be started at boot time, the package also installs a startup script in */etc/rc.d*. If I install the OpenLDAP server, the package installation will report:

```
...
The following new rcscripts were installed: /etc/rc.d/slapd
```

To start the `slapd(8)` OpenLDAP server at boot, add the script name to the `pkg_scripts` variable in */etc/rc.conf.local*.

```
pkg_scripts="slapd"
```

OpenBSD runs these scripts in order at boot, and in reverse order at shutdown.

To change a package's command-line arguments from the default, add a *command*_flags variable to */etc/rc.conf.local*. Do not edit the startup script.

```
slapd_flags="-u _openldap -6 -l local0"
```

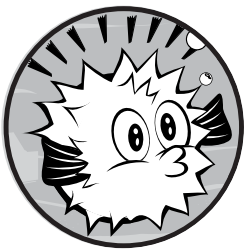
You can now manage your add-on software in any way you need.

Now let's move on to configuring OpenBSD's integrated software, through the files in */etc*.

14

EVERYTHING /ETC

You are in a maze
of twisty little configs;
no two are alike.



When I get saddled with an unfamiliar server, the first thing I do is look for the previous sysadmin's documentation. When I discover there isn't any, I study the */etc* directory because it contains the basic configuration for a Unix-like system. The fastest way to go from a junior sysadmin to a mid-grade one is to read */etc* and the associated man pages—all of the documentation. Yes, that's a lot of reading.

Once you understand */etc*, you understand how the system hangs together. You must learn all this anyhow, but you might as well take the easier route and learn it up front rather than in an unscheduled series of desperate troubleshooting sessions. (I've already discussed many */etc* files in earlier chapters where relevant, such as */etc/fstab* in Chapter 8 and */etc/services* in Chapter 11.)

Some files that you'll find in */etc* are of only historical interest or are gradually dying out. I'll discuss briefly what they do, but I won't spend much time on obsolete files. I also won't spend much time on files useful only in

edge cases (where they relate to software that's not used too often or only in very peculiar circumstances). On the other hand, I will dive deeply into important */etc* files that haven't already found a place elsewhere in this book.

/etc Across Unix Variants

Different Unix-like systems use different */etc* files. In many commercial variants, these files are simply renamed or restructured files from primordial BSD.

For example, the first time I had to manage an IBM AIX system, I needed to know which disks should be mounted where, but there was no */etc/fstab*. A little searching led me to */etc/filesystems*, which was IBM's */etc/fstab*. Apparently, the IBM folks felt that a file named for "filesystem table" was confusing, so they gave it a different name. Knowing that this information existed somewhere in */etc*, and knowing which files had an unrelated purpose, greatly shortened my search.

Changes to */etc* can dramatically alter how your system performs. While manually recovering a scrambled filesystem can push an adequate sysadmin toward becoming a pretty good one, it's one of the least pleasant ways to get there.

The /etc Files

Now we will take a look at each of the */etc* files, going in alphabetical order. We'll start with */etc/adduser.conf* and end with */etc/ypldap.conf*.

/etc/adduser.conf

The */etc/adduser.conf* file contains the persistent configuration for `adduser(8)`, which Chapter 6 covers in detail. To change `adduser`'s settings, edit this file. This file is self-documenting, and if you've read Chapter 6, you shouldn't have any trouble with it.

/etc/amd

The automounter daemon automatically loads NFS filesystems upon request. If you're interested in this function, read the `amd(8)` man page.

The automounter daemon isn't as useful as you might think. As the documentation says, "A weird imagination is most useful to gain full advantage of all the features."

/etc/authpf

The */etc/authpf* directory contains the authenticating packet filter configuration. If you're interested in having user authentication for firewall access, I recommend *The Book of PF, 2nd Edition*, by Peter Hansteen (No Starch Press, 2010).

/etc/bgpd.conf

OpenBSD includes a BGP daemon, `bgpd(8)`. Most sysadmins will never go near BGP, but if you need it, see the `bgpd.conf(5)` man page.

/etc/boot.conf

The */etc/boot.conf* file controls the system's booting process, as discussed in Chapter 5.

/etc/changelist

/etc/changelist is a text file that contains the list of files to be checked by the daily security check script, */etc/security*. We cover these tasks in detail in Chapter 15.

/etc/chio.conf

The `chio(1)` medium changer lets you manage jukebox-style media arrays, such as CD and tape changers. If you have a tape backup unit that can swap between multiple tapes all on its own, `chio` is your friend. Configure `chio` in */etc/chio.conf*. Most OpenBSD users don't have media changers, but if you're interested, see the man page for `chio(1)`.

/etc/csh.*

The */etc/csh.** files contain system-wide defaults for `csh(1)` and `tcsh(1)`. When a user logs in with either of these shells, the shell executes any commands listed in */etc/csh.login*. Similarly, when the user logs out, the shell executes any commands in */etc/csh.logout*. Put general shell configuration in */etc/csh .cshrc*.

/etc/daily and /etc/daily.local

The */etc/daily.local* script is run every day to maintain the system. See Chapter 15 for details.

/etc/dhclient.conf

You can configure OpenBSD's DHCP client by using `dhclient(8)`, as discussed in Chapter 12.

/etc/dhcpd.conf

OpenBSD includes a secure DHCP server. It started life as an Internet Systems Consortium (ISC) DHCP server, but has been repeatedly rewritten and simplified by the OpenBSD team. See Chapter 16 for details.

/etc/disklabels/

The */etc/disklabels/* directory traditionally contains backup copies of disklabels, as discussed in Chapter 8. Very few system administrators actually

use it, because very few system administrators know it's there. OpenBSD copies the disklabels for all system disks as part of the daily backup of critical files (see Chapter 15).

/etc/disktab

A couple of decades ago, hard drives were expensive and precious devices that came in only a few varieties. While modern disks can tell the computer about their geometry, older disks need manual configuration, as do older removable media such as 1.44MB floppy disks and Zip disks. If you want to access one of these ancient disks, you'll need the information in */etc/disktab*. (But if you need to use one of these old disks, you're almost certainly solving the wrong problem.)

/etc/dumpdates

The */etc/dumpdates* file records the dates of the last successful `dump(8)` backup. The `dump` tool backs up a filesystem, rather than just the files on a filesystem, as `tar(1)` does. A dump includes file flags and other special characteristics of the filesystem.

/etc/dvmrpd.conf

OpenBSD implements the Distance Vector Multicast Routing Protocol (DVMRP) with `dvmrpd(8)`, configured in *dvmrpd.conf*. This is another edge case where OpenBSD performs well, but is of interest only to a few users. If you're interested in DVMRP, see the `dvmrpd(8)` man pages.

/etc/exports

NFS servers list the filesystems they export, and who may access them, in */etc/exports*. See Chapter 9 for information about using NFS as both a client and a server.

/etc/fbtab

OpenBSD can change the ownership of system files and devices based on how a user has logged in. When a user logs in via the text console, `login(1)` changes the ownership of the console, as well as the keyboard and the mouse, to that user. This way, users can then configure the keyboard or mouse to suit their preferences. A user who logs in via an X session needs similar changes.

You could use */etc/fbtab* to change permissions on other devices or files for other special needs, but this is really tricky to get right, and usually means you're making something much more complicated than it needs to be. If you're considering this approach, rethink your problem.

/etc/firmware

Some hardware needs software to run. In years past, this software would have been loaded onto programmable read-only memory (PROM) chips

on the device itself. Network cards often had boot PROMs that let them get configuration information from the network. PROMs are not expensive these days, but making millions of anything adds up quickly. Hardware vendors now often ask the operating system to load this software, called *firmware*, onto the hardware for them.

Firmware is usually a closed-source binary object, or *blob*. While the OpenBSD team members would never accept loading a blob into the kernel, they're willing to hand a file to a piece of hardware. It doesn't matter if the hardware gets the blob from the operating system or from a PROM chip attached to the hardware—it's still software that runs only on the device, not in the operating system.

For user convenience, OpenBSD distributes firmware that it's legally permitted to offer. If OpenBSD cannot distribute firmware on the installation media, the `fw_update` script uses the package system to fetch it over the Internet. Wherever the firmware comes from, it's stored in */etc/firmware*.

By default, `fw_update` runs at first boot after an OpenBSD install or upgrade, but you can run it any time you want (it's stored in */usr/sbin*).

/etc/fonts/

The */etc/fonts/* directory contains the Xenocara X11 fonts. We'll discuss Xenocara and X11 in Chapter 17.

/etc/fstab

The filesystem table lists all filesystems that the system knows about, whether they're automatically mounted at boot or kept in reserve. See Chapter 8 for details.

/etc/ftpchroot

Users listed in */etc/ftpchroot* are automatically `chroot`ed into their home directory when they log in via FTP. See `ftpd(8)` for full details.

Try to avoid FTP, as it transmits usernames and passwords in cleartext. Use `scp(1)` or `sftp(1)` instead.

/etc/ftpusers

The */etc/ftpusers* file does exactly the opposite of what the filename implies, but it has been this way for decades, so don't worry about it.

Any user listed in */etc/ftpusers* cannot log in via FTP. See `ftpd(8)` for details, but again, use `scp(1)` or `sftp(1)` instead—especially for administrative tasks!

/etc/gettytab

The */etc/gettytab* file contains the configuration information for login terminals. Unix-like systems have been traditionally accessed by anything and everything, from innumerable slightly different serial consoles to Secure Shell (SSH) sessions over the network. If you ever need to use a nonstandard terminal type, explore */etc/gettytab*.

There's no reason to get rid of this file. After all, someone out there probably still uses a plugboard 1200-baud terminal. But there's no longer any reason to modify it either. In any case, the man page is worth reading, especially if you read it to the end.

/etc/group

The */etc/group* file controls to which groups each user account belongs. User groups are covered in Chapters 6 and 7.

/etc/hostapd.conf

OpenBSD can act as a wireless access point. The host access point daemon (`hostapd(8)`) lets OpenBSD perform some complicated access point tasks that are useful in larger environments. OpenBSD's wireless services are interesting if you use wireless; read `hostapd(8)` and `ifconfig(8)` for full details.

/etc/hostname.*

The */etc/hostname.** files configure network interfaces, as discussed briefly in Chapter 4 and at length in Chapter 12.

/etc/hosts

The */etc/hosts* file contains a hand-maintained map of IP addresses and hostnames. See Chapter 12 for details.

/etc/hosts.equiv

The */etc/hosts.equiv* file is used by the various r-tools, such as `rcp(1)`, `rlogin(1)`, and `rsh(1)`. These tools, and the underlying protocols, are relics of an earlier age, when security on the Internet was not so great a concern. OpenBSD no longer includes an `rlogin(1)` client. The r-tools are largely replaced by SSH. Today, you should not use the r-protocols under any sensible circumstances.

The */etc/hosts.equiv* file should not contain any uncommented entries, unless you specifically put them there. This is the only system file I can reasonably say every sysadmin should verify is empty and marked immutable (see Chapter 10).

What did I mean by "sensible circumstances" and using the r-protocols? A few years ago, I worked on a network with ancient but mission-critical VMS servers manageable only by r-protocols, but those were not reasonable circumstances. If you find yourself similarly trapped, check out `hosts.equiv(5)`, `rshd(8)`, and `rsh(1)`. And do something about your circumstances.

/etc/hosts.lpd

You can configure an OpenBSD system to accept print requests from other machines and feed them to a locally attached printer. This was vital when

printers were big, expensive beasts, but it's less important today. This file lists the hostnames or IP addresses of systems that may use the local system's line printer daemon, `lpd(8)`.

If you're really interested in having your OpenBSD system print, check out `lpd(8)` and */etc/printcap*.

/etc/hotplug/

OpenBSD can automatically take action when a device is plugged into the system through `hotplugd(8)`. For example, if you attach your digital camera to a USB port, `hotplugd` can run a script that will attach the device node to a directory and make it readable by your user account.

`hotplugd` runs the script */etc/hotplug/attach* when a device is attached, and */etc/hotplug/detach* when a device is removed. These scripts must be carefully written to match the devices being attached and detached. For details on how this works, see `hotplugd(8)`.

/etc/ifstated.conf

The interface state daemon `ifstated(8)` monitors network conditions and takes action when specified events occur. For example, you can configure `ifstated` to watch for another server to go down, and when that server fails, start up the local web server. For more information about using `ifstated`, see Hansteen's *The Book of PF*.

/etc/iked/, /etc/iked.conf, /etc/ipsec.conf, and /etc/isakmpd

The */etc/iked/*, */etc/iked.conf*, */etc/ipsec.conf*, and */etc/isakmpd* files manage OpenBSD's implementation of the IPsec standard for VPNs. OpenBSD has a very robust IPsec implementation, and it is actually used for testing by various interoperability groups.

You can fill entire books with the acronyms used for VPN technologies, let alone instructions for actually configuring them. See the man pages for details.

/etc/inetd.conf

The Internet small service listener `inetd(8)` attaches to multiple network ports for programs that don't need to run constantly. When a request on one of those ports comes in, `inetd` activates the correct program to handle the request. Chapter 16 discusses `inetd(8)` in more detail.

/etc/kbdtype

The */etc/kbdtype* file contains a single line with your preferred terminal keyboard mapping. Test your keyboard mapping with `kdb(1)`, and then put that keyboard mapping in */etc/kbdtype*.

/etc/kerberosV/

The */etc/kerberosV/* directory contains configuration for the Kerberos centralized authentication system. I recommend some kind of centralized logon for every network. Configuring Kerberos properly is a large subject. If you're interested, peruse `kerberos(1)`.

/etc/ksh.kshrc

/etc/ksh.kshrc is the global configuration file for the public domain Korn shell included with OpenBSD. Users must include this file in their personal *.kshrc* for settings here to take effect. See the `ksh(1)` man page for details.

/etc/ldap/ and /etc/ldapd.conf

OpenBSD includes a Lightweight Directory Access Protocol (LDAP) daemon. LDAP is commonly used for centralized authentication, address books, and other database operations that are read more than they write. As of this writing, the features of `ldapd(8)` are not complete, but it's good enough to provide a central authentication point.

/etc/localtime

The */etc/localtime* file is a symlink to the actual time zone file. To change the time zone, change the symlink. Chapter 4 covers time zones.

/etc/locate.rc

The */etc/locate.rc* file controls creation of the `locate(1)` database. OpenBSD updates the locate database every week, as discussed in Chapter 15.

/etc/login.conf

With */etc/login.conf*, you can control user account login behavior, as discussed in Chapter 6.

/etc/lynx.cfg

OpenBSD includes the `lynx(1)` text-mode web browser. Lynx is endlessly configurable, and settings in */etc/lynx.cfg* affect all Lynx users on the system. You can save yourself a lot of trouble by configuring your proxy server settings here.

/etc/magic

Many files include a "magic number" and other characteristics specific to their type. `file(1)` uses these magic numbers to identify the file type, with */etc/magic* as an index of magic numbers. Do not manually edit */etc/magic*, as it's automatically generated by compiling `file(1)`.

/etc/mail/

The */etc/mail/* directory contains the configuration files for OpenBSD's email software. OpenBSD includes two email packages: the old workhorse Sendmail and the OpenBSD-created `smptd(8)`. `smptd` isn't *quite* ready for production use, but it might be by the time this book hits the shelves.

This directory also contains */etc/mail/aliases*, a list of mail redirections. Be sure you set the email alias for your root account to somewhere you'll actually read your email, as discussed in Chapter 4.

/etc/mail.rc

The */etc/mail.rc* file has nothing to do with sending or receiving email as a mail server. It's the global configuration file for the `mail(1)` email client. While more advanced email clients have almost completely superseded `mail`, it's worth exploring because almost any Unix-like system will have it installed.

/etc/mailer.conf

Traditionally, the only mail server program available for any Unix-like operating system was Sendmail. As such, a huge amount of add-on software expects to find */usr/sbin/sendmail* and expects it to behave in a certain manner. It doesn't matter that, by modern standards, Sendmail and the whole Simple Mail Transfer Protocol (SMTP) are baroque and bizarre; software expects to find it.

Worse, `sendmail(8)` behaved differently depending on what name it was called with. Programs such as `send-mail`, `mailq`, and `newaliases` are all Sendmail wearing different hats. If you call the Sendmail program by running the `mailq` command, it runs differently than if you call it by running the `newaliases` command. They all point to the same binary on disk, however. Third-party software expects to find all these names as well, and that these commands behave as required.

Sendmail is such a standard that writers of newer mail server programs are forced to call them `sendmail` and to have them behave exactly as Sendmail does, just to maintain compatibility with this vast installed base. This makes using alternate mailers confusing. Also, OpenBSD includes Sendmail as part of the base system. You can't just remove Sendmail and go on. Upgrades reinstall brand-name Sendmail.

OpenBSD does an end run around all this confusion by eliminating */usr/sbin/sendmail* as an actual mail program. Instead, the `sendmail` program is just a wrapper that calls the real mail-handling software. The entries in */etc/mailer.conf* are just a list of classic Sendmail program names, along with the path to the actual program to be run. The mail-handling program `sendmail` is actually installed as */usr/libexec/sendmail/sendmail*, for example.

To run an alternate mail server, give */etc/mailer.conf* the expected command name and the full path to all of the appropriate programs. This happens automatically when you install a new mail transfer agent from a package.

/etc/man.conf

The *etc/man.conf* file tells `man(1)` how to find and present man pages. If you install software in nonstandard locations, add the information on the software's man pages to *man.conf* so that you can call up those man pages transparently. This file has several types of entries, each of which is set off by keywords or section names.

Why would you do this? If you must install software from a source other than ports or packages, you could place it in a directory tree outside those managed by OpenBSD to reduce confusion when upgrading or adding software.

For example, I occasionally install Wireshark on an OpenBSD desktop. The OpenBSD team decided to remove OpenBSD's Wireshark package because it has such a shaky security history. If I installed Wireshark as */usr/local/bin*, it would be mingled with my packaged software. If I install Wireshark as */usr/local/wireshark/bin*, however, it's clearly distinct from packaged software. I can't access the man pages, however, as `man(1)` doesn't know about */usr/local/wireshark/man*.[1] Let's walk through how we would add man page access for Wireshark.

Adding to the Search Index

The `_whatdb` keyword gives the full path to a `whatis(1)` database, used by `apropos(1)` and `whatis(1)`, allowing you to easily search and cross-index man pages. The file *man.conf* has entries for databases in */usr/local/man*, */usr/share/man*, and */usr/X11R6/man*. Here's how to add an entry for */usr/local/wireshark/man*:

```
_whatdb /usr/local/wireshark/man/whatis.db
```

Now we create a new database with `makewhatis(8)`. This job runs automatically when you install software and during weekly maintenance.

Adding to Man Page Directories

Man pages are scattered in directories all over the system. The `_default` keyword tells `man(1)` which directories to search automatically. Use only one `_default` keyword, but list multiple directories if needed, and group directories using brackets.

Here's the standard directory list:

```
_default /usr/{share, X11R6,local,ports/infrastructure}/man/
```

This is a pretty massive group of directories, and the brackets combine multiple directories associatively. For example, this entry means that we

1. If Wireshark has a shaky security history, why would I install it? Because, sadly, it's still the easiest way to debug really complicated network problems, especially when you're working with an unfamiliar protocol.

check */usr/share/man/*, */usr/X11R6/man/*, */usr/local/man/*, and */usr/local/ports/infrastructure/man/*. (The entries end with a slash, which means that the final directories contain subdirectories.)

To add */usr/local/wireshark/man/* to the default locations, add `local/wireshark` into the associative array, like this:

```
_default /usr/{share, X11R6,local,ports/infrastructure,local/wireshark}/man/
```

The `_subdir` keyword lists subdirectories to be searched beneath the main directories, in order, with the first match returned first.

```
_subdir  man1 man8 man6 man2 man3 man5 man7 man4 man9 man3p man3f mann
```

You can use `_subdir` to change the order in which man pages are returned. For example, if your job is programming Perl on OpenBSD, you might want to see the Perl man pages by default. In that case, you could move the *man3p* subdirectory to the front of the list.

Displaying Man Pages

Software vendors distribute their manuals in the format they think best. As a sysadmin, this means you might as well get manuals in random formats, because each format needs a different command to display it. Fortunately, each format has a different filename suffix, which tells `man(1)` how to display the file. The `_build` keyword defines a filename suffix and the command used to display the file. (It's very unlikely that you'll have a man page that requires a new `_build` statement.)

Defining Man Sections

The final *man.conf* function is dividing the manual into sections. We saw in Chapter 1 that you can search the manual by particular sections to get specific man pages. These sections are nothing more than directories identified in */etc/man.conf*. Here's where we define the man pages included in section 1.

```
1 /usr/{share,X11R6,local}/man/man1
```

There's no trailing slash, because we're not adding any subdirectories. These are the actual directories containing section 1 of the manual.

You can define arbitrary section names in */etc/man.conf*. While you should avoid section names beginning with an underscore in order to prevent confusion with keywords, you can do just about anything else.

/etc/master.passwd, /etc/passwd, /etc/spwd.db, and /etc/pwd.db

The */etc/master.passwd*, */etc/passwd*, */etc/spwd.db*, and */etc/pwd.db* files contain usernames and passwords, along with a few other key items about